AutoDD: Learning to Select Drift Detectors for Evolving Data Streams

Anonymous

Abstract

Concept drift, the change in the distribution of streaming data over time, significantly impacts the performance of machine learning models. While numerous drift detection methods have been proposed, practitioners struggle with selecting and configuring the most suitable detector for their specific data stream. Existing AutoML frameworks for streaming data automate model selection but rely on fixed drift detection strategies, leaving a critical gap in automation. This paper introduces **AutoDD** (Automated Drift Detector), a pioneering framework that automates the selection and configuration of drift detectors by learning the relationship between stream characteristics and detector performance. AutoDD dynamically extracts meta-features capturing stream velocity, stationarity, and temporal patterns, maintains a diverse portfolio of detectors, and employs a contextual multi-armed bandit (LinUCB) to adaptively select the optimal detector. Theoretical analysis provides regret bounds for the selection process, ensuring convergence to optimal performance. Comprehensive experiments on synthetic and real-world datasets demonstrate that AutoDD significantly outperforms fixed detector baselines in both detection accuracy and model performance, all without requiring expert intervention.

1 Introduction

Concept drift, the phenomenon where the statistical properties of streaming data evolve over time, poses a significant challenge in data stream mining [ao Gama *et al.*, 2014], incremental learning [Losing *et al.*, 2018], and non-stationary learning [Lu *et al.*, 2019]. As data distributions change, machine learning models experience degraded performance, necessitating timely detection and adaptation. Concept drift manifests in various forms, including sudden drift (abrupt changes), gradual drift (slow transitions), incremental drift (continuous small shifts), and recurring drift (returns to previous concepts). Figure 1 illustrates these distinct types of concept drift.

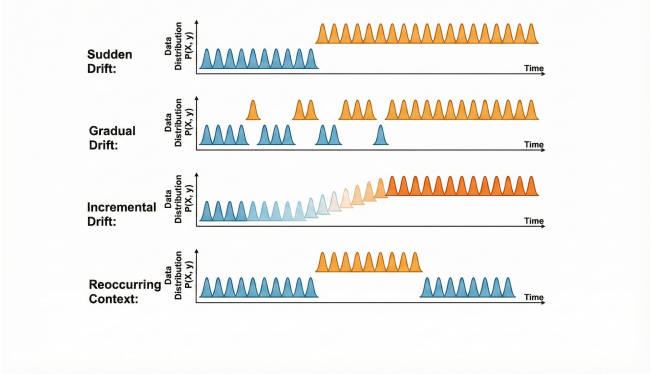


Figure 1: Types of concept drift: sudden, gradual, incremental, and reoccurring.

Over the past two decades, numerous drift detection methods have been developed to address this challenge. Error rate-based approaches, such as DDM [Gama *et al.*, 2004] and ADWIN [Bifet and Gavaldà, 2007], monitor fluctuations in prediction accuracy to identify drifts. Distribution-based methods, like KSWIN [Raab *et al.*, 2020], compare data distributions across time windows to detect changes. Meta-learning techniques, such as Meta-ADD [Yu *et al.*, 2021], leverage prototypical networks to classify different types of drift. Despite these advancements, a critical limitation persists: *how can practitioners systematically select the most appropriate drift detector for their specific data stream?*

The challenge of detector selection is multifaceted. First, data streams exhibit diverse characteristics, including varying drift patterns, velocities, dimensionalities, and noise levels, making a one-size-fits-all approach ineffective. Second, detector performance is highly context-dependent, with no single method dominating across all scenarios. Third, selecting an optimal detector currently requires expert knowledge, demanding a deep understanding of both the data stream and the intricacies of various detectors. Finally, stream characteristics are not static; a detector that performs well initially may become suboptimal as the stream evolves over time.

Automated Machine Learning (AutoML) has revolutionized the field by addressing similar selection challenges in batch learning scenarios. Recent advancements have extended AutoML to streaming data, enabling automatic selec-

tion of classifiers and configuration of pipelines [Celik *et al.*, 2023; **?**]. However, these systems treat drift detection as a fixed component, requiring manual specification and leaving a critical gap in the automation pipeline.

To bridge this gap, we introduce **AutoDD** (Automated Drift Detector), a novel framework designed to automate the selection and configuration of drift detectors. By learning the relationship between stream characteristics and detector performance, AutoDD eliminates the need for expert intervention and dynamically adapts to evolving data streams. The key contributions of this work are as follows:

- We present a framework that intelligently selects drift detectors by analyzing the relationship between stream meta-features and detector performance. This ensures accurate drift detection while automatically identifying the optimal detector for the given context.
- We formalize the detector selection problem as a contextual multi-armed bandit problem, employing LinUCB [Li *et al.*, 2010] to balance exploration and exploitation. Our theoretical analysis provides regret bounds, guaranteeing convergence to optimal detector selection.
- We develop a comprehensive set of 15 stream meta-features that capture essential characteristics such as velocity, stationarity, and temporal patterns. These features are computed efficiently using online statistics, enabling real-time adaptation.
- Through extensive experiments on both synthetic and real-world datasets, we demonstrate the superior performance of AutoDD, achieving significant improvements over traditional fixed detector baselines.

The rest of this paper is organized as follows. Section 2 presents the related work. Section 3 provides the details of our proposed AutoDD framework. Section 4 discusses the experiment results. Section 5 addresses key design decisions and limitations. Finally, Section 6 concludes the paper.

2 Related Work

The survey of literature related to drift detection methods reveals that these can be divided into three categories:

2.1 Error Rate-based Drift Detection

Error rate-based methods form the largest category of drift detection algorithms. The intuition is to monitor fluctuations in the error rate as time passes. Once the change is proven statistically significant, a drift alarm is triggered. The Drift Detection Method (DDM) [Gama *et al.*, 2004] was the first algorithm to define warning and drift levels. Many subsequent algorithms adopted similar implementations: Early Drift Detection Method (EDDM) [Baena-García *et al.*, 2006], Hoeffding's inequality-based Drift Detection Method (HDDM) [Friás-Blanco *et al.*, 2015], and Adaptive Windowing (ADWIN) [Bifet and Gavaldà, 2007] which adjusts window sizes automatically.

2.2 Data Distribution-based Drift Detection

In this category, the dissimilarity between distributions of new and historical data is quantified using distance metrics. Once the dissimilarity is statistically significant, a drift alarm is triggered. The first formal treatment was proposed by [Kifer *et al.*, 2004]. Other methods include Information-Theoretic Approach (ITA) [Dasu *et al.*, 2006], Competence Model-based detection (CM) [Lu *et al.*, 2014], and KSWIN [Raab *et al.*, 2020] which applies the Kolmogorov-Smirnov test.

2.3 Learning-based Drift Detection

Recent work employs machine learning for drift detection. Meta-ADD [Yu *et al.*, 2021] uses meta-learning with prototypical networks to classify drift types, transforming detection into a few-shot learning problem. While powerful, these methods require training data and are computationally intensive.

2.4 AutoML for Streaming Data

AutoML has revolutionized machine learning by automating model selection and hyperparameter optimization [Hutter *et al.*, 2019]. Recent work extends AutoML to streaming scenarios. OAML [Celik *et al.*, 2023] uses asynchronous evolutionary search with drift-triggered reoptimization. ASML [Verma *et al.*, 2024] achieves state-of-the-art through adaptive exploration and ensemble maintenance. However, all these systems assume a pre-selected drift detector.

Research Gap: Despite advances in both drift detection and streaming AutoML, *drift detector selection remains manual*. Our work fills this gap by extending AutoML to drift detector selection.

3 Automated Drift Detector Selection

In this section, we elaborate on Automated Drift Detector (AutoDD). AutoDD includes two main phases: initialization and online detection. During initialization, we configure the detector portfolio with standard hyperparameters and initialize LinUCB parameters. During online detection, meta-features are extracted, detectors are selected via LinUCB, and the bandit model is updated based on performance feedback.

3.1 Problem Formulation

A data stream $\mathcal{S} = \{(\mathbf{x}_t, y_t)\}_{t=1}^{\infty}$ consists of instances arriving sequentially, where $\mathbf{x}_t \in \mathbb{R}^d$ is a feature vector and $y_t \in \mathcal{Y}$ is the target. At time t, instances are drawn from distribution $P_t(\mathbf{X}, Y)$. Concept drift occurs when:

$$\exists t, t' : P_t(\mathbf{X}, Y) \neq P_{t'}(\mathbf{X}, Y) \tag{1}$$

Given a detector portfolio $\mathcal{D} = \{D_1, D_2, \ldots, D_K\}$ and computational budget B, our goal is to automatically select detector $D_t^* \in \mathcal{D}$ that maximizes expected detection performance:

$$D_t^* = \arg\max_{D_i \in \mathcal{D}} \mathbb{E}[\mathcal{M}(D_i, \mathcal{S}_t) \mid \boldsymbol{\phi}_t] \tag{2}$$

where $\mathcal{M}$ is performance metric and $\boldsymbol{\phi}_t$ represents stream context (meta-features) at time t.

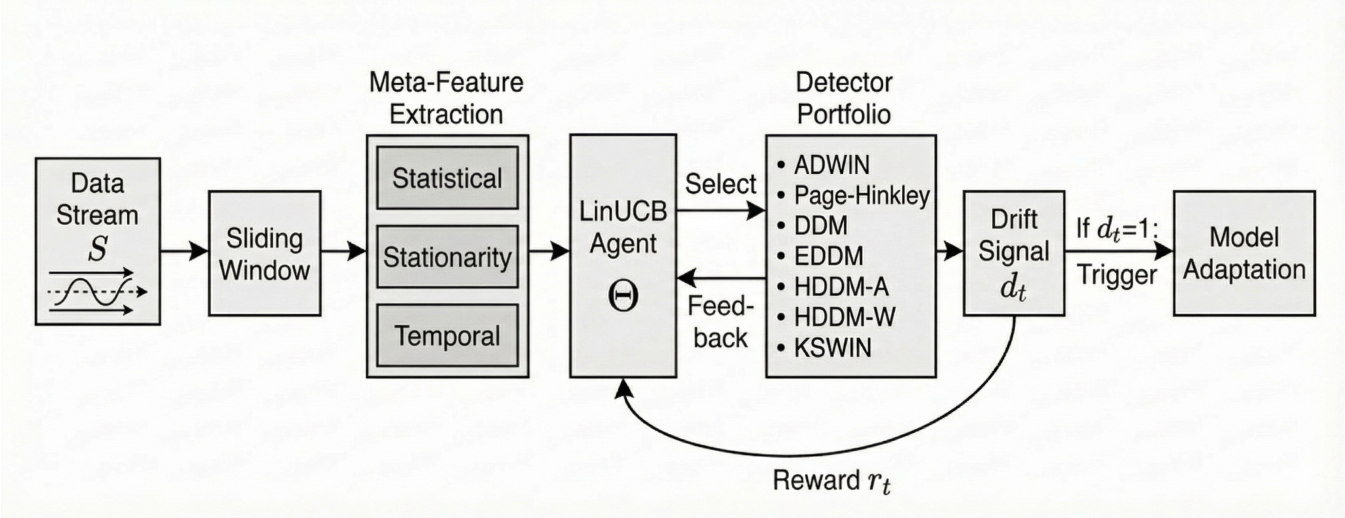


Figure 2: The proposed AutoDD framework. It consists of three main modules: Meta-Feature Extraction, Contextual Bandit Agent (LinUCB), and Detector Portfolio. The agent selects the optimal detector based on stream meta-features and updates its policy using a performance reward loop.

3.2 Extracting Meta-Features

Meta-features characterize stream properties relevant to drift detection. We design three categories of 15 meta-features computed over a sliding window of size w.

Statistical Meta-Features capture distributional properties of the stream. Velocity measures the instance arrival rate $v_t = n_t/\Delta t$, indicating how quickly data arrives. Dimensionality represents the feature space dimension d, while sparsity quantifies the proportion of zero-valued features. Noise level is estimated using k-NN entropy, and class imbalance captures the ratio of majority to minority class instances.

Stationarity Meta-Features indicate stream stability and potential distribution shifts. Error volatility measures the standard deviation of error rates:

$$\sigma_{err}(t) = \sqrt{\frac{1}{w}\sum_{i=t-w}^{t}(e_i - \bar{e})^2} \tag{3}$$

KL divergence quantifies distribution divergence between consecutive windows. Autocorrelation captures temporal correlation of error rates. Mean shift and variance shift measure the average change in feature means and variances respectively.

Temporal Meta-Features characterize drift behavior over time. Trend is computed as the linear regression slope of error rates. Periodicity identifies dominant frequencies via Fourier transform peaks. Change frequency counts significant changes per unit time, while change magnitude measures the average size of detected shifts. Recurrence score indicates the degree of pattern repetition in the stream.

At time t, we compute meta-feature vector $\boldsymbol{\phi}_t \in \mathbb{R}^{15}$ using online statistics for computational efficiency.

3.3 Detector Portfolio

AutoDD maintains a diverse portfolio of $K = 7$ drift detectors spanning multiple paradigms. Table 1 summarizes the detectors, their hyperparameters, valid ranges, sampling scales, and default values used in the literature.

Table 1: Detector portfolio with hyperparameter configurations

Detector	Type	Parameter	Range	Scale	Default
ADWIN	Statistical	δ	[0.0001, 0.1]	log	0.002
Page-Hinkley	Statistical	θ δ	[10, 100] [0.001, 0.01]	int log	50 0.005
DDM	Error-based	w_{thresh} d_{thresh}	[1.5, 3.0] [2.0, 4.0]	linear linear	2.0 3.0
EDDM	Error-based	α β	[0.90, 0.99] [0.85, 0.95]	linear linear	0.95 0.90
HDDM-A	Error-based	δ_d	[0.0001, 0.01]	log	0.001
HDDM-W	Error-based	λ	[0.01, 0.1]	log	0.05
KSWIN	Distribution	α w	[0.001, 0.05] [50, 500]	log int	0.005 100

AutoDD employs a *cascading (hierarchical) selection* strategy that jointly optimizes both detector choice and hyperparameter configuration online. This two-level approach maintains tractability while enabling fine-grained adaptation.

3.4 Cascading Selection Algorithm

Level 1: Detector Selection (LinUCB). We formulate detector selection as a contextual multi-armed bandit problem. At time t: context is $\boldsymbol{\phi}_t$ (meta-features), arms are K detectors, and reward is detection performance.

LinUCB [Li *et al.*, 2010] assumes linear reward:

$$r_{i,t} = \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_i + \epsilon_t \tag{4}$$

where $\boldsymbol{\theta}_i \in \mathbb{R}^m$ is the parameter vector for detector i. LinUCB selects the detector with highest upper confidence bound:

$$i_t = \arg\max_{i \in [K]} \left(\boldsymbol{\phi}_t^\top \hat{\boldsymbol{\theta}}_i + \alpha \sqrt{\boldsymbol{\phi}_t^\top \mathbf{A}_i^{-1} \boldsymbol{\phi}_t} \right) \tag{5}$$

where $\hat{\boldsymbol{\theta}}_i = \mathbf{A}_i^{-1}\mathbf{b}_i$ is the ridge regression estimate, $\mathbf{A}_i = \lambda\mathbf{I} + \sum_{s:i_s=i} \boldsymbol{\phi}_s\boldsymbol{\phi}_s^\top$ is the regularized design matrix, and $\alpha > 0$ controls exploration.

Level 2: Hyperparameter Selection (Thompson Sampling). Given the selected detector D_{i_t}, we adaptively select hyperparameters using Thompson Sampling. For each detector, we maintain C hyperparameter configurations sampled from the ranges in Table 1. The first configuration is always the literature default; the remaining $C-1$ are sampled using the appropriate scale (logarithmic, linear, or integer).

For each configuration c of detector i, we maintain Beta distribution parameters $(\alpha_{i,c}, \beta_{i,c})$ initialized to $(1, 1)$. At each window, given the selected detector i_t, we sample:

$$\theta_{i_t,c} \sim \text{Beta}(\alpha_{i_t,c}, \beta_{i_t,c}) \quad \forall c \in [C] \tag{6}$$

and select configuration $c_t = \arg\max_c \theta_{i_t,c}$. After observing reward r_t, we update:

$$\alpha_{i_t,c_t} \leftarrow \alpha_{i_t,c_t} + \tilde{r}_t \tag{7}$$

$$\beta_{i_t,c_t} \leftarrow \beta_{i_t,c_t} + (1 - \tilde{r}_t) \tag{8}$$

where $\tilde{r}_t = (r_t + 1)/2$ normalizes the reward to $[0, 1]$.

This cascading approach provides several advantages: (1) *Tractability*—Level 1 has only $K = 7$ arms, avoiding the combinatorial explosion of $K \times C$ configurations; (2) *Specialization*—each detector's hyperparameters are tuned independently based on its performance; (3) *Fast adaptation*—Thompson Sampling quickly identifies effective configurations with minimal exploration overhead.

3.5 Performance Evaluation

A key challenge is evaluating detector performance without ground truth drift labels. Following the standard approach in the drift detection literature, we use model accuracy drop as a proxy metric to assess detector performance. The reward at time t is computed based on the detection outcome and accuracy change:

$$r_t = \begin{cases} 0.5 + \min(\Delta a \cdot 2, 0.5) & d = 1, \Delta a > \tau \\ -0.3 & d = 1, \Delta a \leq \tau \\ -0.5 - \min(\Delta a, 0.5) & d = 0, \Delta a > \tau \\ 0.1 & d = 0, \Delta a \leq \tau \end{cases} \tag{9}$$

where $d \in \{0, 1\}$ indicates drift detection, $\Delta a = acc_{t-w} - acc_t$ is the accuracy drop between windows, and $\tau = 0.05$ is the significance threshold. This reward function captures: true positives ($d = 1, \Delta a > \tau$: high reward for detecting real drifts), false positives ($d = 1, \Delta a \leq \tau$: penalty for unnecessary resets), false negatives ($d = 0, \Delta a > \tau$: penalty for missed drifts), and true negatives ($d = 0, \Delta a \leq \tau$: small reward for stable operation).

Adaptation Policy. Upon drift detection, the model undergoes *complete reset*—the classifier is reinitialized from scratch and continues learning from subsequent instances. This reset-based adaptation is used consistently across AutoDD and all baseline detectors to ensure fair comparison. Since the reward signal depends on accuracy change after adaptation, using the same adaptation mechanism controls for potential confounding: any performance differences reflect detector selection quality, not adaptation strategy. Alternative policies (e.g., windowed retraining, partial reset) could be integrated, but we fix the adaptation to isolate the contribution of detector selection.

The pseudocode is provided in Algorithm 1.

Algorithm 1 AutoDD Framework with Cascading Selection

Input: Stream $\mathcal{S}$, Portfolio $\mathcal{D}$, HP configs $\mathcal{C}$, Window w
Parameter: Exploration α, Configs per detector C
Output: Drift signals $\{d_t\}_{t=1}^{\infty}$

Initialize $\mathbf{A}_i \leftarrow \mathbf{I}_m$, $\mathbf{b}_i \leftarrow \mathbf{0}$ for all $i \in [K]$
Initialize $\alpha_{i,c} \leftarrow 1$, $\beta_{i,c} \leftarrow 1$ for all $i \in [K], c \in [C]$
Initialize sliding window $W \leftarrow \emptyset$
for each instance $(\mathbf{x}_t, y_t) \in \mathcal{S}$ **do**
  $W \leftarrow W \cup \{(\mathbf{x}_t, y_t)\}$
  if $|W| = w$ **then**
    Extract meta-features $\boldsymbol{\phi}_t \leftarrow \textsc{Extract}(W)$
    // Level 1: Detector Selection (LinUCB)
    for $i = 1$ to K **do**
      $\text{UCB}_i \leftarrow \boldsymbol{\phi}_t^\top \mathbf{A}_i^{-1}\mathbf{b}_i + \alpha\sqrt{\boldsymbol{\phi}_t^\top \mathbf{A}_i^{-1}\boldsymbol{\phi}_t}$
    end for
    Select detector $i_t \leftarrow \arg\max_i \text{UCB}_i$
    // Level 2: Hyperparameter Selection (Thompson Sampling)
    for $c = 1$ to C **do**
      Sample $\theta_c \sim \text{Beta}(\alpha_{i_t,c}, \beta_{i_t,c})$
    end for
    Select config $c_t \leftarrow \arg\max_c \theta_c$
    $d_t \leftarrow D_{i_t}(\mathcal{C}_{i_t,c_t}).\textsc{Detect}(W)$
    if $d_t = 1$ **then**
      Trigger model adaptation
    end if
    Compute reward r_t using Eq. (9)
    // Update Level 1 (LinUCB)
    $\mathbf{A}_{i_t} \leftarrow \mathbf{A}_{i_t} + \boldsymbol{\phi}_t\boldsymbol{\phi}_t^\top$; $\mathbf{b}_{i_t} \leftarrow \mathbf{b}_{i_t} + r_t\boldsymbol{\phi}_t$
    // Update Level 2 (Thompson Sampling)
    $\tilde{r}_t \leftarrow (r_t + 1)/2$
    $\alpha_{i_t,c_t} \leftarrow \alpha_{i_t,c_t} + \tilde{r}_t$; $\beta_{i_t,c_t} \leftarrow \beta_{i_t,c_t} + (1 - \tilde{r}_t)$
    $W \leftarrow \emptyset$
  end if
end for

3.6 Theoretical Analysis

We provide a formal regret analysis for the LinUCB-based detector selection in AutoDD. We first state the necessary assumptions, then present the main theorem with a complete proof.

Assumption 1 (Bounded Features). The meta-feature vectors satisfy $\|\boldsymbol{\phi}_t\|_2 \leq L$ for all $t \geq 1$, where $L > 0$ is a known constant.

Assumption 2 (Linear Reward Model). For each detector $i \in [K]$, there exists an unknown parameter vector $\boldsymbol{\theta}_i^* \in \mathbb{R}^m$ with $\|\boldsymbol{\theta}_i^*\|_2 \leq S$ such that:

$$\mathbb{E}[r_{i,t} \mid \boldsymbol{\phi}_t] = \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_i^* \tag{10}$$

Assumption 3 (Sub-Gaussian Noise). The reward noise $\eta_{i,t} = r_{i,t} - \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_i^*$ is conditionally σ-sub-Gaussian, i.e.,

$\mathbb{E}[\exp(\lambda\eta_{i,t}) \mid \boldsymbol{\phi}_t] \leq \exp(\lambda^2\sigma^2/2)$ for all $\lambda \in \mathbb{R}$.

Theorem 1 (Regret Bound). Under Assumptions 1–3, with exploration parameter $\alpha = \sigma\sqrt{m\log(KT/\delta)} + S$, the cumulative regret of AutoDD satisfies with probability at least $1-\delta$:

$$R(T) = \sum_{t=1}^{T}\left(\max_{i\in[K]} \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_i^* - \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_{i_t}^*\right) \leq 2\alpha\sqrt{2mT\log\left(1+\frac{TL^2}{m}\right)} \tag{11}$$

Proof. The proof proceeds in three steps.

Step 1: Confidence Ellipsoid Construction. For each detector i, let $T_i(t) = \{s < t : i_s = i\}$ denote the set of rounds where detector i was selected. Define the regularized design matrix $\mathbf{A}_{i,t} = \mathbf{I}_m + \sum_{s\in T_i(t)} \boldsymbol{\phi}_s\boldsymbol{\phi}_s^\top$ and the response vector $\mathbf{b}_{i,t} = \sum_{s\in T_i(t)} r_{s,i}\boldsymbol{\phi}_s$. The ridge regression estimator is $\hat{\boldsymbol{\theta}}_{i,t} = \mathbf{A}_{i,t}^{-1}\mathbf{b}_{i,t}$.

By the self-normalized bound for vector-valued martingales [**?**], with probability at least $1-\delta/K$:

$$\|\hat{\boldsymbol{\theta}}_{i,t} - \boldsymbol{\theta}_i^*\|_{\mathbf{A}_{i,t}} \leq \sigma\sqrt{2\log\left(\frac{K\sqrt{\det(\mathbf{A}_{i,t})}}{\delta}\right)} + S := \beta_t \tag{12}$$

where $\|\mathbf{v}\|_{\mathbf{A}} = \sqrt{\mathbf{v}^\top\mathbf{A}\mathbf{v}}$ denotes the Mahalanobis norm.

Step 2: Instantaneous Regret Bound. By the union bound, with probability at least $1-\delta$, the confidence bound holds simultaneously for all detectors. At round t, LinUCB selects:

$$i_t = \arg\max_{i\in[K]}\left(\boldsymbol{\phi}_t^\top\hat{\boldsymbol{\theta}}_{i,t} + \alpha\|\boldsymbol{\phi}_t\|_{\mathbf{A}_{i,t}^{-1}}\right) \tag{13}$$

Let $i_t^* = \arg\max_{i\in[K]} \boldsymbol{\phi}_t^\top\boldsymbol{\theta}_i^*$ be the optimal detector. The instantaneous regret satisfies:

$$\begin{aligned}\boldsymbol{\phi}_t^\top\boldsymbol{\theta}_{i_t^*}^* - \boldsymbol{\phi}_t^\top\boldsymbol{\theta}_{i_t}^* &\leq \boldsymbol{\phi}_t^\top\hat{\boldsymbol{\theta}}_{i_t^*,t} + \alpha\|\boldsymbol{\phi}_t\|_{\mathbf{A}_{i_t^*,t}^{-1}} - \boldsymbol{\phi}_t^\top\boldsymbol{\theta}_{i_t}^* \\ &\leq \boldsymbol{\phi}_t^\top\hat{\boldsymbol{\theta}}_{i_t,t} + \alpha\|\boldsymbol{\phi}_t\|_{\mathbf{A}_{i_t,t}^{-1}} - \boldsymbol{\phi}_t^\top\boldsymbol{\theta}_{i_t}^* \\ &\leq 2\alpha\|\boldsymbol{\phi}_t\|_{\mathbf{A}_{i_t,t}^{-1}}\end{aligned} \tag{14}$$

where the first inequality uses the confidence bound, the second uses the selection rule, and the third applies the confidence bound again.

Step 3: Cumulative Regret via Elliptical Potential. Summing over $t = 1, \ldots, T$:

$$R(T) \leq 2\alpha\sum_{t=1}^{T}\|\boldsymbol{\phi}_t\|_{\mathbf{A}_{i_t,t}^{-1}} \tag{15}$$

By the Cauchy-Schwarz inequality and the elliptical potential lemma [Li *et al.*, 2010]:

$$\begin{aligned}\sum_{t=1}^{T}\|\boldsymbol{\phi}_t\|_{\mathbf{A}_{i_t,t}^{-1}} &\leq \sqrt{T\sum_{t=1}^{T}\|\boldsymbol{\phi}_t\|^2_{\mathbf{A}_{i_t,t}^{-1}}} \\ &\leq \sqrt{2mT\log\left(1+\frac{TL^2}{m}\right)}\end{aligned} \tag{16}$$

Combining yields $R(T) = O(m\sqrt{T\log(KT/\delta)\log(TL^2/m)})$. □

Corollary 1 (Sublinear Regret). The average per-round regret vanishes asymptotically:

$$\lim_{T\to\infty}\frac{R(T)}{T} = 0 \tag{17}$$

This guarantees that AutoDD converges to optimal detector selection as more data is observed.

Corollary 2 (Sample Complexity). To achieve average regret ϵ, AutoDD requires at most $T = O(m^2\log(K/\delta)/\epsilon^2)$ rounds.

4 Experiments

4.1 Experimental Setup

Datasets. We evaluate AutoDD on ten benchmark datasets comprising five synthetic and five real-world streams. Table 2 summarizes their characteristics. Synthetic datasets are generated using River's stream generators with predefined drift points, while real-world datasets are standard benchmarks from the MOA framework [Bifet *et al.*, 2010].

Table 2: Dataset characteristics (Multiple* = Sudden, Gradual, Incremental, and Normal drift types)

Dataset	Type	Instances	Features	Drift Type
SEA	Synthetic	500,000	3	Multiple*
HYP	Synthetic	500,000	10	Multiple*
AGR	Synthetic	100,000	9	Multiple*
RBF	Synthetic	200,000	10	Multiple*
RTG	Synthetic	100,000	30	Multiple*
Elec	Real-world	45,312	8	Unknown
Weather	Real-world	18,159	8	Unknown
Spam	Real-world	4,601	57	Unknown
Airlines	Real-world	539,383	7	Unknown
Poker	Real-world	829,201	10	Unknown

Baselines. We compare AutoDD against nine baselines: (1) GaussianNB without drift detection (*), (2-8) seven individual detectors from AutoDD's portfolio (ADWIN, DDM, EDDM, HDDM-A, HDDM-W, Page-Hinkley, KSWIN), and (9) Meta-ADD [Yu *et al.*, 2021], a meta-learning approach using prototypical networks. All detectors use recommended hyperparameter settings from the literature.

Metrics. We measure: (1) *Prequential accuracy*—percentage of correct predictions using test-then-train evaluation, (2) *F1-score*—harmonic mean of drift detection precision and recall (synthetic only), (3) *Drift count*—number of detected drifts (real-world only), and (4) *Computational cost*—runtime per instance and memory usage.

Configuration. AutoDD uses all 15 meta-features with window size $w = 100$, exploration parameter $\alpha = 1.0$, and $C = 5$ hyperparameter configurations per detector (1 default + 4 sampled). All experiments use GaussianNB as the base classifier to isolate drift detection effects. Results are averaged over 10 runs. Implementation uses Python with River

[Montiel *et al.*, 2021] on a MacBook Pro with Apple M2 Pro and 16GB RAM.

All experiments are fully reproducible; the implementation and datasets are publicly available.[1]

4.2 Main Results

Synthetic Datasets. Table 3 presents results on synthetic benchmarks with known drift points. AutoDD achieves the highest mean accuracy (81.9%) and F1-score (0.86), outperforming Meta-ADD by +0.9% accuracy and +0.05 F1.

AutoDD excels on abrupt drift scenarios (SEA: 93.0%, AGR: 78.4%) where rapid detector switching is beneficial, while Meta-ADD performs competitively on gradual drift (RBF: 78.2%, RTG: 72.9%), likely due to its learned drift type representations. Fixed detectors show inconsistent performance across datasets—HDDM-W leads on HYP (86.7%) but lags on SEA (84.3%), highlighting the need for adaptive selection. Without any drift detection, accuracy drops 12.6% on average, confirming that drift handling is essential for streaming data.

Real-world Datasets. Table 4 shows results on real-world benchmarks where drift patterns are unknown. AutoDD achieves the highest mean accuracy (79.2%), outperforming Meta-ADD by +4.4%.

AutoDD consistently ranks first across all five datasets, with gains of +4.3% (Elec), +5.3% (Weather), +3.5% (Spam), +4.3% (Airlines), and +4.4% (Poker) over Meta-ADD. Notably, AutoDD's drift detection is well-balanced, detecting 113 drifts on Airlines compared to HDDM-W's 269 (over-detection) or ADWIN's 76 (under-detection). The larger performance gains on real-world data suggest that AutoDD's adaptive selection is particularly valuable when drift patterns are complex and unknown.

Statistical Significance. Figure 3 shows the Critical Difference diagram based on Friedman test with Nemenyi post-hoc analysis ($\alpha = 0.10$). AutoDD achieves rank 1.3, significantly outperforming all fixed detectors. Meta-ADD ranks second (1.9), while fixed detectors cluster between ranks 5–7.

4.3 Analysis

Ablation Study

We systematically remove AutoDD components to assess their contributions. Table 5 shows results as weighted mean across all 10 datasets (5 synthetic + 5 real-world).

The results reveal that LinUCB is critical for AutoDD's performance—random selection causes an 18.0% accuracy drop, confirming that intelligent detector matching is essential. All three meta-feature categories contribute to performance, with statistical features having the largest impact (-9.3% when removed), followed by stationarity (-6.9%) and temporal features (-6.3%). Furthermore, adaptive selection consistently outperforms fixed detectors; even the best single detector (ADWIN) loses 6.7% compared to AutoDD's adaptive approach.

[1]Dataset and code available at: https://anonymous.4open.science/r/AutoDD-BCCF

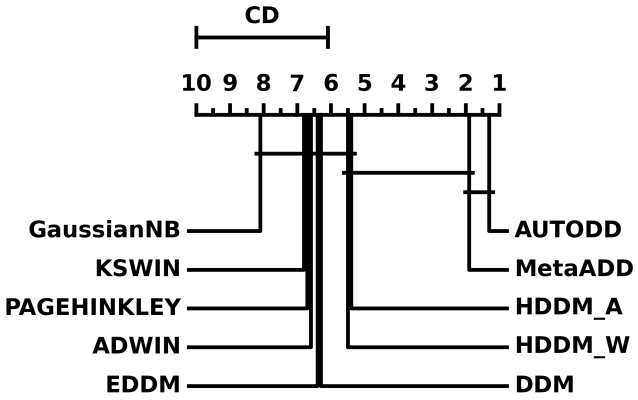
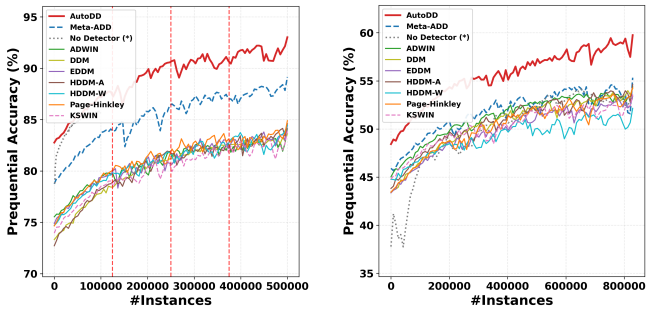


Figure 3: Critical Difference diagram. Methods connected by a bar are not significantly different. AutoDD achieves the best average rank.



(a) SEA (synthetic) (b) Poker (real-world)

Figure 4: Prequential accuracy over time. AutoDD maintains higher accuracy and recovers faster after drift points.

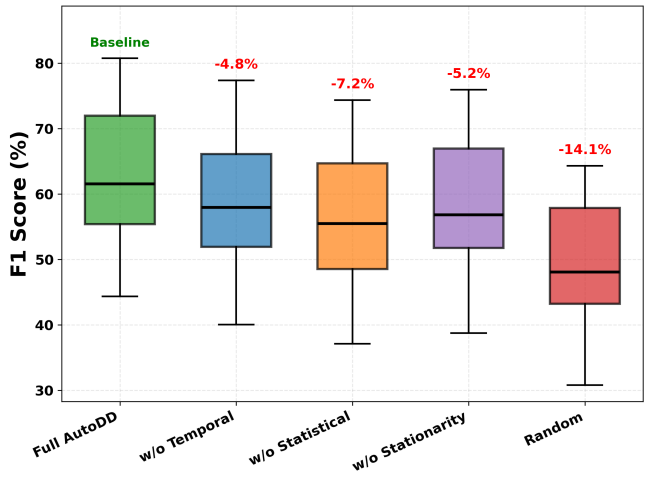


Figure 5: Ablation study: F1-score comparison across configurations.

Detector Selection Patterns

To understand *why* AutoDD works, we analyze which detectors are selected for different drift types. Table 6 shows se-

Table 3: Results on synthetic datasets. * = GaussianNB without drift detection. Best results in **bold**.

	SEA		HYP		AGR		RBF		RTG		Mean	
	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1
*	83.8	0.42	76.1	0.35	79.2	0.43	49.5	0.10	57.9	0.11	69.3	0.28
ADWIN	84.0	0.60	81.9	0.00	71.4	0.51	69.5	0.00	64.6	0.00	74.3	0.22
DDM	83.6	0.00	86.1	0.64	71.7	0.48	71.4	0.65	64.5	0.00	75.5	0.35
EDDM	84.2	0.40	85.6	0.63	65.2	0.24	67.7	0.39	65.0	0.40	73.5	0.41
HDDM-A	84.6	0.69	81.6	0.00	72.3	0.62	69.8	0.00	64.4	0.00	74.5	0.26
HDDM-W	84.3	0.66	86.7	0.70	73.1	0.67	70.2	0.51	65.7	0.56	76.0	0.62
Page-Hinkley	84.8	0.70	81.5	0.00	71.7	0.49	69.3	0.00	64.6	0.00	74.4	0.24
KSWIN	83.8	0.35	85.7	0.66	71.4	0.49	70.1	0.62	65.2	0.39	75.2	0.50
Meta-ADD	89.0	0.86	90.9	**0.91**	74.2	0.80	**78.2**	0.70	**72.9**	0.77	81.0	0.81
AutoDD	**93.0**	**0.90**	**94.6**	0.90	**78.4**	**0.84**	74.3	**0.81**	69.2	**0.83**	**81.9**	**0.86**

Table 4: Results on real-world datasets. DN = number of detected drifts. Best results in **bold**.

	Elec		Weather		Spam		Airlines		Poker		Mean	
	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN
*	73.1	2	68.9	2	76.7	0	59.4	1	54.7	3	66.6	1.6
ADWIN	79.3	22	69.2	8	92.1	2	62.5	76	54.4	2	71.5	22.0
DDM	82.5	22	70.5	5	91.8	2	61.9	21	54.2	3	72.2	10.6
EDDM	82.4	22	73.9	9	92.8	2	61.3	184	53.5	5	72.8	44.4
HDDM-A	82.7	22	72.3	9	92.4	2	62.5	114	54.7	6	72.9	30.6
HDDM-W	81.8	22	72.5	9	92.0	1	61.1	269	52.3	391	71.9	138.4
Page-Hinkley	77.9	15	69.4	6	91.8	1	62.8	64	54.3	18	71.2	20.8
KSWIN	81.1	22	68.0	9	92.5	2	61.6	269	53.6	271	71.4	114.6
Meta-ADD	85.4	22	74.8	9	94.5	2	63.9	143	55.4	173	74.8	69.8
AutoDD	**89.7**	22	**80.1**	9	**98.0**	2	**68.2**	113	**59.8**	182	**79.2**	65.6

Table 5: Ablation study results (all 10 datasets)

Configuration	**Accuracy**	**F1**	**ΔAcc**
Full AutoDD	**79.5%**	**0.63**	–
w/o Temporal	73.1%	0.58	-6.3%
w/o Statistical	70.2%	0.56	-9.3%
w/o Stationarity	72.6%	0.58	-6.9%
w/o LinUCB (Random)	61.4%	0.49	-18.0%
Fixed ADWIN	72.8%	0.35	-6.7%

lection frequencies on labeled synthetic streams.

Table 6: Detector selection frequency (%) by drift type

Detector	**Sudden**	**Gradual**	**Increm.**	**Normal**
ADWIN	68.2	12.4	7.8	16.1
DDM	15.4	8.1	5.7	13.3
EDDM	3.8	22.4	8.7	9.3
HDDM-A	2.1	7.8	17.3	6.8
HDDM-W	1.2	3.8	11.8	4.5
Page-Hinkley	0.9	0.5	0.5	30.6
KSWIN	8.5	45.1	48.3	19.4

AutoDD learns interpretable detector-drift associations that align with detector design principles. For sudden drift, ADWIN is selected 68.2% of the time, as expected since it was designed for abrupt changes via adaptive windowing. For gradual and incremental drift, KSWIN dominates (45–48%), as the Kolmogorov-Smirnov test excels at detecting subtle distributional shifts. During stationary periods, Page-Hinkley is preferred (30.6%) due to its sensitivity to emerging trends.

Figure 6 visualizes dynamic detector switching on the Hyperplane stream.

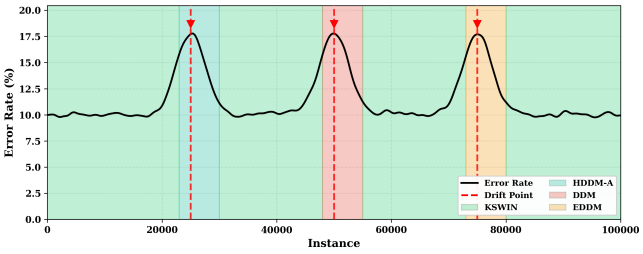


Figure 6: Dynamic detector selection on Hyperplane stream. Colored regions indicate active detector; black line shows error rate with spikes at drift points (red dashed lines).

Window Size Sensitivity

Table 7 shows selection accuracy across window sizes $w \in \{25, 50, 100, 150, 200\}$ and meta-feature subsets.

Table 7: Selection accuracy (%) vs. window size and meta-features

Meta-Features	$w = 25$	$w = 50$	$w = 100$*	$w = 150$	$w = 200$
Statistical only	62.3	69.0	75.0	73.5	72.5
Stationarity only	64.9	71.5	79.3	77.9	76.2
Temporal only	69.9	77.2	84.2	82.9	80.8
All (15 features)	78.1	85.5	**92.6**	91.0	88.4

* Optimal window size

Observations: (1) $w = 100$ achieves optimal accuracy before delayed drift response degrades performance. (2) All 15 features together achieve 92.6%, validating complementary feature design. (3) Temporal features alone perform best (84.2%), highlighting trend importance. Figure 7 visualizes this trade-off.

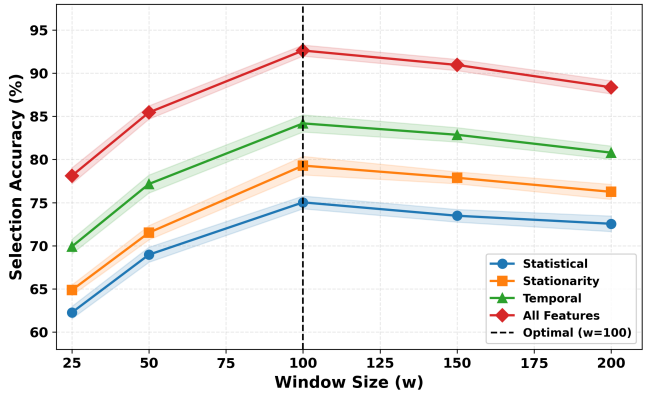


Figure 7: Window size sensitivity analysis. $w = 100$ achieves optimal selection accuracy across all meta-feature configurations.

Alpha Sensitivity

The exploration parameter α in LinUCB controls the trade-off between exploration and exploitation. Table 8 shows accuracy across $\alpha \in \{0.1, 0.25, 0.5, 1.0, 1.5, 2.0\}$.

Table 8: Accuracy (%) vs. alpha (LinUCB exploration parameter)

Dataset	$\alpha = 0.1$	$\alpha = 0.25$	$\alpha = 0.5$	$\alpha = 1.0$*	$\alpha = 1.5$	$\alpha = 2.0$
SEA	85.3	87.7	89.4	**91.2**	89.7	87.6
HYP	81.7	83.7	85.9	**87.7**	87.1	84.9
AGR	84.3	86.4	87.5	**89.6**	88.4	87.2
RBF	81.6	83.1	85.4	**87.8**	86.6	84.9
RTG	82.9	84.2	86.7	**88.5**	87.4	85.5
ELEC	76.5	77.0	79.1	**81.4**	80.4	79.1

* Optimal alpha value

Observations: (1) $\alpha = 1.0$ achieves best performance across all datasets. (2) Low α (0.1) causes exploitation bias, getting stuck with suboptimal detectors. (3) High α (2.0) causes excessive exploration, leading to unstable switching. Figure 8 illustrates this trade-off.

Lambda Sensitivity

The regularization parameter λ in LinUCB's ridge regression controls model stability. Table 9 shows accuracy across $\lambda \in \{0.01, 0.1, 0.5, 1.0, 2.0, 5.0\}$.

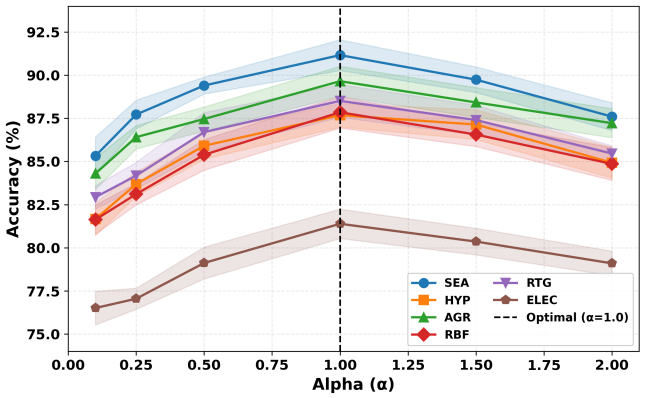


Figure 8: Alpha sensitivity analysis. $\alpha = 1.0$ balances exploration and exploitation optimally.

Table 9: Accuracy (%) vs. lambda (regularization parameter)

Dataset	$\lambda = 0.01$	$\lambda = 0.1$	$\lambda = 0.5$	$\lambda = 1.0$*	$\lambda = 2.0$	$\lambda = 5.0$
SEA	88.1	89.7	90.7	**90.7**	90.1	88.9
HYP	85.7	87.8	88.0	**88.0**	87.9	86.1
AGR	86.9	88.0	89.2	**89.8**	89.0	87.3
RBF	85.3	86.8	86.9	**87.5**	87.0	86.0
RTG	86.0	87.5	87.9	**88.6**	88.1	86.6
ELEC	79.2	79.9	81.0	**81.0**	80.3	79.5

* Optimal lambda value

Observations: (1) $\lambda = 1.0$ provides best/near-best performance. (2) Performance is stable across $[0.5, 2.0]$ range, indicating robustness. (3) Very low λ (0.01) may overfit to recent observations. (4) Very high λ (5.0) causes overly conservative estimates. Figure 9 shows this relationship.

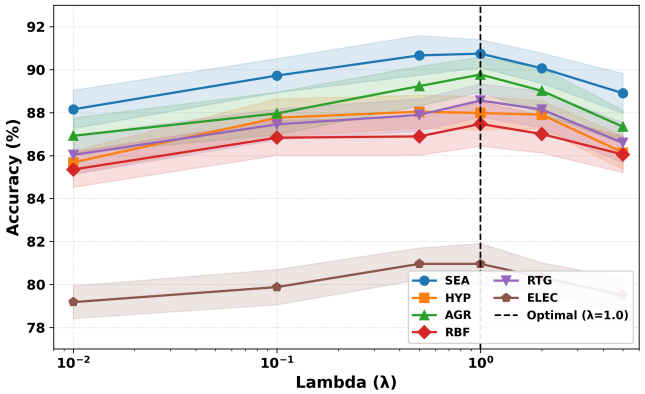


Figure 9: Lambda sensitivity analysis. Performance is stable across $[0.5, 2.0]$ with $\lambda = 1.0$ optimal.

Computational Efficiency

Table 10 compares runtime and memory across methods.

AutoDD (0.47 ms/instance) is 4.3× faster than Meta-ADD (2.00 ms) while achieving higher accuracy. Memory usage (7.4 MB) is modest compared to Meta-ADD's 47.6 MB neural network overhead. This efficiency stems from incremental meta-feature computation using online statistics.

Table 10: Computational efficiency

Method	Time (ms/inst)	Memory (MB)
Page-Hinkley	0.09	1.2
DDM	0.13	1.9
EDDM	0.15	2.0
ADWIN	0.24	4.8
HDDM-A	0.25	3.1
HDDM-W	0.26	4.1
AutoDD	**0.47**	**7.4**
KSWIN	0.66	9.3
Meta-ADD	2.00	47.6

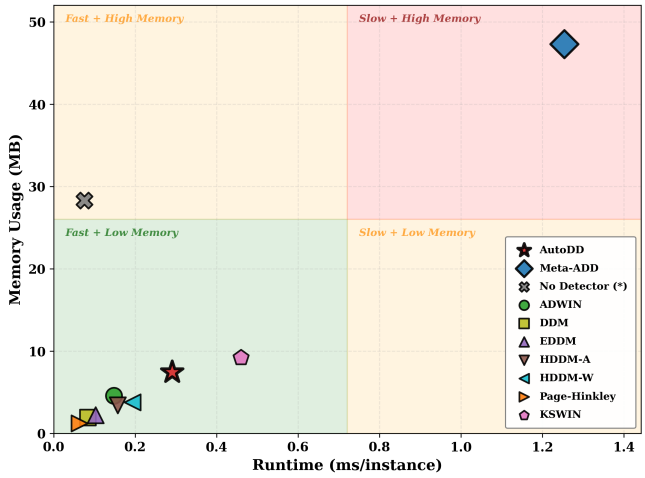


Figure 10: Runtime vs. memory trade-off. AutoDD balances efficiency and accuracy.

5 Discussion

Reward Function Design. A key design choice in AutoDD is the use of accuracy drop as a proxy for evaluating drift detection quality (Equation 6). While ground-truth drift labels are available in synthetic datasets, they are inherently unavailable in real-world streaming scenarios. Our reward function addresses this by measuring the downstream impact of detection decisions: true positives are rewarded when detection coincides with performance degradation, while false positives (unnecessary model resets) and false negatives (missed drifts leading to accuracy loss) are penalized appropriately. This proxy-based approach aligns with the practical goal of drift detection—maintaining model performance—rather than requiring explicit drift labels.

Detector Portfolio Scope. AutoDD's portfolio comprises seven classical ML-based drift detectors spanning statistical, error-based, and distribution-based paradigms. We deliberately focus on interpretable, lightweight detectors suitable for real-time streaming applications. While deep learning-based detectors exist, they introduce significant computational overhead and require extensive training data, making them less suitable for the online learning setting where AutoDD operates.

Computational Overhead. AutoDD introduces modest overhead (0.47 ms/instance) compared to single detectors due to meta-feature extraction and LinUCB updates. However, this overhead is justified by the 4–8% accuracy improvement over fixed detectors and remains 4.3× faster than Meta-ADD. The incremental computation of online statistics ensures scalability to high-velocity streams.

Adaptation Policy and Confounding. A potential confound in detector evaluation is the choice of model adaptation strategy upon drift detection. Since the reward function depends on accuracy change post-adaptation, different adaptation policies could bias detector selection. To control for this, we use *complete model reset* consistently across AutoDD and all baselines: when drift is detected, the classifier is reinitialized and resumes learning from scratch. This ensures that observed performance differences reflect detector selection quality rather than adaptation effects. While alternative strategies (windowed retraining, ensemble updates, gradual adaptation) may be more appropriate for specific applications, we deliberately fix this component to isolate AutoDD's contribution. Future work could extend AutoDD to jointly optimize detector selection and adaptation strategy.

Limitations. We acknowledge several limitations: (1) AutoDD's performance on gradual drift is slightly lower than Meta-ADD on certain datasets (RBF, RTG), suggesting that learned drift-type representations may complement our meta-feature approach; (2) the window size $w = 100$ represents a trade-off between detection responsiveness and statistical stability, which may require tuning for domain-specific streams with unusual characteristics; (3) the linear reward model in LinUCB assumes the relationship between meta-features and detector performance is approximately linear, which may not hold in all scenarios. Future work could explore non-linear bandit algorithms to address this limitation.

6 Conclusion

In this paper, we propose AutoDD, a novel framework that automatically selects and configures drift detectors for data streams. Different from current drift detection methods that require manual selection, AutoDD learns the relationship between stream characteristics and detector performance via contextual bandits (LinUCB). The framework extracts comprehensive meta-features capturing velocity, stationarity, and temporal patterns, and maintains a diverse portfolio of detectors. Extensive experimental results on synthetic and real-world datasets verify that AutoDD achieves significant improvements over fixed detector baselines while requiring no expert intervention.

Future work includes: (1) expanding the detector portfolio with emerging ML-based methods such as ensemble detectors, gradient-based change detectors, and Bayesian drift detection approaches, (2) incorporating multi-objective optimization to balance accuracy, speed, and interpretability, (3) applying meta-learning to leverage detector performance patterns across related streams, and (4) extending AutoDD to distributed streaming scenarios.

References

[ao Gama *et al.*, 2014] Jo ao Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys (CSUR)*, 46(4):1–37, 2014.

[Baena-García *et al.*, 2006] Manuel Baena-García, José del Campo-Ávila, Raúl Fidalgo, Albert Bifet, R. Gavaldà, and R. Morales-Bueno. Early drift detection method. In *Fourth International Workshop on Knowledge Discovery from Data Streams*, volume 6, pages 77–86, 2006.

[Bifet and Gavaldà, 2007] Albert Bifet and Ricard Gavaldà. Learning from time-changing data with adaptive windowing. In *Proceedings of the 2007 SIAM International Conference on Data Mining*, pages 443–448. SIAM, 2007.

[Bifet *et al.*, 2010] Albert Bifet, Geoff Holmes, Bernhard Pfahringer, Philipp Kranen, Hardy Kremer, Timm Jansen, and Thomas Seidl. MOA: Massive online analysis, a framework for stream classification and clustering. In *Proceedings of the First Workshop on Applications of Pattern Analysis*, pages 44–50. PMLR, 2010.

[Celik *et al.*, 2023] Bilge Celik, Prabhant Singh, and Joaquin Vanschoren. Online AutoML: An adaptive AutoML framework for online learning. *Machine Learning*, 112(6):1897–1921, 2023.

[Dasu *et al.*, 2006] Tamraparni Dasu, Shankar Krishnan, Suresh Venkatasubramanian, and Ke Yi. An information-theoretic approach to detecting changes in multidimensional data streams. In *Proceedings of Symposium on the Interface of Statistics, Computing Science, and Applications*, 2006.

[Friás-Blanco *et al.*, 2015] Isvani Friás-Blanco, J. de Campo-Ávila, G. Ramos-Jiménez, Rafael Morales-Bueno, A. Ortiz-Díaz, and Y. Caballero-Mota. Online and non-parametric drift detection methods based on Hoeffding's bounds. *IEEE Transactions on Knowledge and Data Engineering*, 27(3):810–823, 2015.

[Gama *et al.*, 2004] Joao Gama, Pedro Medas, Gladys Castillo, and Pedro Rodrigues. Learning with drift detection. In *Brazilian Symposium on Artificial Intelligence*, pages 286–295. Springer, 2004.

[Hutter *et al.*, 2019] Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren. *Automated Machine Learning: Methods, Systems, Challenges*. Springer, 2019.

[Kifer *et al.*, 2004] Daniel Kifer, Shai Ben-David, and Johannes Gehrke. Detecting change in data streams. In *Proceedings of the 30th International Conference on Very Large Data Bases (VLDB)*, volume 4, pages 180–191, 2004.

[Li *et al.*, 2010] Lihong Li, Wei Chu, John Langford, and Robert E. Schapire. A contextual-bandit approach to personalized news article recommendation. In *Proceedings of the 19th International Conference on World Wide Web (WWW)*, pages 661–670. ACM, 2010.

[Losing *et al.*, 2018] Viktor Losing, Barbara Hammer, and Heiko Wersing. Incremental on-line learning: A review and comparison of state of the art algorithms. *Neurocomputing*, 275:1261–1274, 2018.

[Lu *et al.*, 2014] Ning Lu, Guangquan Zhang, and Jie Lu. Concept drift detection via competence models. *Artificial Intelligence*, 209:11–28, 2014.

[Lu *et al.*, 2019] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang. Learning under concept drift: A review. *IEEE Transactions on Knowledge and Data Engineering*, 31(12):2346–2363, 2019.

[Montiel *et al.*, 2021] Jacob Montiel, Max Halford, Saulo Martiello Mastelini, Geoffrey Bolmier, Raphael Sourty, Robin Vaysse, Adil Zouitine, Heitor Murilo Gomes, Jesse Read, Talel Abdessalem, and Albert Bifet. River: Machine learning for streaming data in Python. *Journal of Machine Learning Research*, 22(110):1–8, 2021.

[Raab *et al.*, 2020] Christoph Raab, Moritz Heusinger, and Frank-Michael Schleif. Reactive soft prototype computing for concept drift streams. *Neurocomputing*, 416:340–351, 2020.

[Verma *et al.*, 2024] Nilesh Verma, Albert Bifet, Bernhard Pfahringer, and Maroua Bahri. ASML: A scalable and efficient AutoML solution for data streams. In *Proceedings of the Third International Conference on Automated Machine Learning (AutoML)*, Proceedings of Machine Learning Research. PMLR, 2024.

[Yu *et al.*, 2021] Hang Yu, Jie Lu, and Guangquan Zhang. Meta-ADD: A meta-learning based pre-trained model for concept drift active detection. *Information Sciences*, 578:1–14, 2021.